Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №4
По дисциплине: «Модели решения задач в И С»
Тема: «Многослойный персептрон для XOR с ReLU на скрытых слоях: сравнение активационных функций»

Выполнил:
Студент 4 курса
Группы ИИ-26
Шоева Е.Т.
Проверил:
Никитин С.К.

Брест 2026

Цель работы: на основе многослойного персептрона p–N–1 решить задачу прогнозирования временного ряда. Научиться формировать обучающую выборку методом скользящего окна, нормализовать данные, оценивать качество одношагового и многошагового прогноза и исследовать влияние размера окна p и числа нейронов скрытого слоя N на точность прогноза и скорость обучения.

Ход работы

Задачи лабораторной работы

1. Сгенерировать ряд по формуле варианта и построить его график с отметкой границы между обучающим и тестовым участками.
2. Нормализовать ряд по минимуму и максимуму **обучающего участка**: x_norm = (x - x_min) / (x_max - x_min). Записать формулу обратного пересчёта и использовать её при вычислении всех метрик в исходной шкале. **Объяснить**, почему min/max нельзя брать по всему ряду.
3. Сформировать обучающую и тестовую выборки методом скользящего окна размера p (вход – p значений, цель – следующее значение). Указать количество получившихся примеров в каждой выборке.
4. Реализовать многослойный персептрон p–N–1:
    - скрытый слой – сигмоида;
    - выходной слой – линейный (задача регрессии), f(s) = s;
    - функция потерь – MSE;
    - обучение – онлайн-режим, перемешивание порядка примеров в каждой эпохе;
    - вывести формулы градиентов для линейного выходного слоя и сигмоидного скрытого слоя и привести их в отчёте.

5. Критерий остановки: средняя по обучающей выборке ошибка $E_s \le E_e$, где $E_e = 10^{-4}$ (нормализованная шкала), либо достижение 20 000 эпох. Скорость обучения подобрать самостоятельно из диапазона 0.01–0.3 и обосновать выбор.
6. **Базовый эксперимент.** Обучить сеть с параметрами p и N из своего варианта. Зафиксировать число эпох, итоговую Es, а также метрики на обучающем и тестовом участках **в исходной шкале**: MAE, RMSE и MAPE (если ряд проходит через ноль – вместо MAPE использовать sMAPE, и пояснить почему).
7. **Исследование окна.** При фиксированном N из варианта обучить сеть для $p \in \{2, 4, p_{вар}, 2 \cdot p_{вар}\}$. Для каждого p выполнить по 5 запусков с разными seed и свести в таблицу: среднее и разброс числа эпох, среднее и разброс тестового MAE, число запусков, не достигших критерия остановки.
8. **Исследование скрытого слоя.** При фиксированном p из варианта обучить сеть для $N \in \{1, 2, N_{вар}, 2 \cdot N_{вар}\}$. Выполнить ту же серию из 5 запусков и заполнить аналогичную таблицу.
9. **Многошаговый прогноз.** Для лучшей по тестовому MAE конфигурации выполнить рекурсивный прогноз на 30 шагов вперёд, начиная с k = 140: каждое предсказание подставляется в окно как входное значение для следующего шага. Построить график «эталон – рекурсивный прогноз» и график абсолютной ошибки в зависимости от номера шага. Определить, на каком шаге ошибка превышает 5 % от размаха ряда.
10. Построить графики:
    - ряд с границей train/test и одношаговым прогнозом сети на обоих участках на одних осях;
    - сходимость (Es от эпохи) для базовой конфигурации и для двух конфигураций из исследования окна на одних осях (логарифмическая шкала по Y);
    - диаграмма рассеяния «эталон – прогноз» на тестовом участке с диагональю y = x;
    - рекурсивный прогноз и рост ошибки по шагам (п. 9).
11. Реализовать режим функционирования: пользователь вводит p последовательных значений ряда в исходной шкале (либо номер k, с которого брать окно из ряда). Программа выводит:

- нормализованные входы;
- прогноз в нормализованной шкале (4 знака после запятой);
- прогноз в исходной шкале;
- истинное значение x(k) и абсолютную ошибку (если окно взято из ряда).

Продемонстрировать работу не менее чем на 3 окнах из обучающего участка, 3 окнах из тестового участка и 1–2 пользовательских окнах (в том числе с «выбросом»).

12. Вставить в отчет фото/скан рукописных расчётов. Вверху листа от руки должны быть написаны Ваша Фамилия, Инициалы, номер группы, вариант и дата выполнения:

- Графическая схема пайплайна (Блок-схема). Составьте корректную цепочку обработки данных, используя предложенный набор блоков (приложение 1). Объясните свой выбор.
- Ручной расчёт 1 шага обучения. На отдельном листе бумаги от руки распишите один полный шаг (Forward + Backward pass) для сети 3–2–1 со следующими параметрами, привязанными к вашему варианту V:
  - Входы: X = [0.2, 0.5, 0.8], Цель: d = 0.6
  - Веса скрытого слоя: W11 = 0.05*V, W12 = -0.2, W21 = 0.3, W22 = 0.05*V, W31 = -0.1, W32 = 0.4 (b1=0.1, b2=-0.1)
  - Веса выходного слоя: W_out1 = 0.5, W_out2 = -0.2*V (b_out = 0.2)

13. Сделать вывод: ответить на все вопросы ниже и проанализировать результаты.

Вариант	**a**	**b**	**c**	**d**	**p**	**N**
11	1.4	0.38	0.7	0.85	6	5

```
import numpy as np
import matplotlib.pyplot as plt

A, B, C, D = 1.4, 0.38, 0.7, 0.85
P_VAR = 6
N_VAR = 5

N_POINTS = 200
TRAIN_END = 140

EE = 1e-4
MAX_EPOCHS = 20000
LEARNING_RATE = 0.1
SEEDS = [1, 2, 3, 4, 5]
BASE_SEED = 1

P_LIST = [2, 4, P_VAR, 2 * P_VAR]
N_LIST = [1, 2, N_VAR, 2 * N_VAR]

FORECAST_STEPS = 30
ERR_THRESHOLD_FRAC = 0.05

NOISE_STD_FRAC = 0.02
NOISE_EPOCHS = 3000

EPS = 1e-12

def generate_series(a=A, b=B, c=C, d=D, n=N_POINTS):
    k = np.arange(n)
    return a * np.cos(b * k) + c * np.sin(d * k)

def fit_minmax(train_part):
    return float(np.min(train_part)), float(np.max(train_part))

def normalize(x, x_min, x_max):
    return (x - x_min) / (x_max - x_min)

def denormalize(x_norm, x_min, x_max):
    return x_norm * (x_max - x_min) + x_min

def make_windows(s, p, start, end):
    X, Y, K = [], [], []
    for t in range(start, end):
        X.append(s[t - p:t][::-1])
        Y.append(s[t])
        K.append(t)
```

```
    return np.array(X), np.array(Y), np.array(K)

def sigmoid(x):
    return 1.0 / (1.0 + np.exp(-x))

class MLP:
    def __init__(self, p, N, lr=LEARNING_RATE, seed=0):
        self.p, self.N, self.lr = p, N, lr
        self.rng = np.random.default_rng(seed)
        self.W1 = self.rng.uniform(-0.5, 0.5, size=(p, N))
        self.b1 = self.rng.uniform(-0.5, 0.5, size=N)
        self.W2 = self.rng.uniform(-0.5, 0.5, size=N)
        self.b2 = float(self.rng.uniform(-0.5, 0.5))

    def forward(self, x):
        h = sigmoid(x @ self.W1 + self.b1)
        y = float(h @ self.W2 + self.b2)
        return h, y

    def predict(self, x):
        return self.forward(np.asarray(x, dtype=float))[1]

    def predict_batch(self, X):
        H = sigmoid(X @ self.W1 + self.b1)
        return H @ self.W2 + self.b2

    def train_step(self, x, d):
        h, y = self.forward(x)
        e = d - y
        delta_h = e * self.W2 * h * (1.0 - h)
        self.W2 += self.lr * e * h
        self.b2 += self.lr * e
        self.W1 += self.lr * np.outer(x, delta_h)
        self.b1 += self.lr * delta_h

    def mean_sq_error(self, X, Y):
        return float(np.mean((Y - self.predict_batch(X)) ** 2))

    def train(self, X, Y, ee=EE, max_epochs=MAX_EPOCHS):
        n = len(X)
        history = []
        Es = float("inf")
        for epoch in range(1, max_epochs + 1):
            for i in self.rng.permutation(n):
                self.train_step(X[i], Y[i])
            Es = self.mean_sq_error(X, Y)
            history.append(Es)
            if Es <= ee:
                return epoch, Es, history, True
        return max_epochs, Es, history, False

def mae(y, y_hat):
    return float(np.mean(np.abs(y - y_hat)))

def rmse(y, y_hat):
    return float(np.sqrt(np.mean((y - y_hat) ** 2)))

def mape(y, y_hat):
    return float(np.mean(np.abs((y - y_hat) / (np.abs(y) + EPS))) * 100.0)

def smape(y, y_hat):
    denom = np.abs(y) + np.abs(y_hat) + EPS
    return float(np.mean(2.0 * np.abs(y - y_hat) / denom) * 100.0)

def all_metrics(y, y_hat):
    return {"MAE": mae(y, y_hat), "RMSE": rmse(y, y_hat), "sMAPE": smape(y, y_hat)}

class Dataset:

    def __init__(self, series, p):
        self.series = series
        self.p = p
        self.x_min, self.x_max = fit_minmax(series[:TRAIN_END])
        self.sn = normalize(series, self.x_min, self.x_max)
        self.Xtr, self.Ytr, self.Ktr = make_windows(self.sn, p, p, TRAIN_END)
        self.Xte, self.Yte, self.Kte = make_windows(self.sn, p, TRAIN_END, len(series))

    def denorm(self, v):
        return denormalize(v, self.x_min, self.x_max)

def run_config(series, p, N, seed, lr=LEARNING_RATE, ee=EE, max_epochs=MAX_EPOCHS):
    ds = Dataset(series, p)
    model = MLP(p, N, lr=lr, seed=seed)
    epochs, Es, history, converged = model.train(ds.Xtr, ds.Ytr, ee=ee, max_epochs=max_epochs)
```

```
    pred_tr = ds.denorm(model.predict_batch(ds.Xtr))
    pred_te = ds.denorm(model.predict_batch(ds.Xte))
    y_tr = ds.denorm(ds.Ytr)
    y_te = ds.denorm(ds.Yte)
    return {
        "p": p, "N": N, "seed": seed, "model": model, "ds": ds,
        "epochs": epochs, "Es": Es, "converged": converged, "history": history,
        "train": all_metrics(y_tr, pred_tr), "test": all_metrics(y_te, pred_te),
        "pred_tr": pred_tr, "pred_te": pred_te, "y_tr": y_tr, "y_te": y_te,
    }

def run_series(series, p, N, seeds=SEEDS, lr=LEARNING_RATE, verbose=True):
    results = []
    for s in seeds:
        r = run_config(series, p, N, s, lr=lr)
        results.append(r)
        if verbose:
            print(f"    p={p:>2} N={N:>2} seed={s}: эпох={r['epochs']:>5} "
                  f"Es={r['Es']:.3e} сошлось={r['converged']} "
                  f"testMAE={r['test']['MAE']:.4f}", flush=True)
    return results

def summarize(results):
    ep = np.array([r["epochs"] for r in results], dtype=float)
    tmae = np.array([r["test"]["MAE"] for r in results])
    trmae = np.array([r["train"]["MAE"] for r in results])
    return {
        "ep_mean": ep.mean(), "ep_std": ep.std(ddof=1), "ep_min": ep.min(), "ep_max": ep.max(),
        "te_mean": tmae.mean(), "te_std": tmae.std(ddof=1), "te_min": tmae.min(), "te_max": tmae.max(),
        "tr_mean": trmae.mean(),
        "n_fail": int(sum(not r["converged"] for r in results)),
        "n": len(results),
    }

def print_study_table(title, key, values, all_results):
    print(f"\n{title}")
    hdr = (f"{key:>4} | {'эпохи (ср.±ст.)':>18} | {'[min; max]':>14} | "
           f"{'train MAE':>9} | {'test MAE (ср.±ст.)':>20} | {'[min; max]':>18} | {'не сошлось':>10}")
    print(hdr)
    print("-" * len(hdr))
    for v, res in zip(values, all_results):
        s = summarize(res)
        print(f"{v:>4} | {s['ep_mean']:>10.1f}±{s['ep_std']:<7.1f} | "
              f"[{s['ep_min']:>5.0f};{s['ep_max']:>6.0f}] | {s['tr_mean']:>9.4f} | "
              f"{s['te_mean']:>11.4f}±{s['te_std']:<8.4f} | "
              f"[{s['te_min']:.4f};{s['te_max']:.4f}] | {s['n_fail']:>4}/{s['n']}")

def lr_sweep(series, lrs=(0.01, 0.03, 0.05, 0.1, 0.2, 0.3)):
    print("\nПодбор скорости обучения (базовая конфигурация p=%d, N=%d, 5 запусков):" % (P_VAR, N_VAR))
    print("«Макс. скачок Es» = max по эпохам Es(t)/Es(t-1): мера амплитуды колебаний в онлайн-режиме.")
    print(f"{'eta':>6} | {'эпох (ср.)':>10} | {'сошлось':>8} | {'test MAE (ср.)':>14} | {'макс. скачок Es':>15}")
    for lr in lrs:
        res = [run_config(series, P_VAR, N_VAR, s, lr=lr) for s in SEEDS]
        sm = summarize(res)
        jump = np.mean([np.max(np.array(r["history"][1:]) / np.array(r["history"][:-1])) for r in res])
        print(f"{lr:>6} | {sm['ep_mean']:>10.1f} | {sm['n'] - sm['n_fail']:>4}/{sm['n']:<3} | "
              f"{sm['te_mean']:>14.4f} | {jump:>14.2f}x")
    print(f"Выбрано eta = {LEARNING_RATE}. Обоснование: при всех eta из диапазона 0.01-0.3 сеть сходится 5/5, "
          f"поэтому выбор — вопрос компромисса. eta=0.01-0.05 требует в 2-10 раз больше эпох (250-1250); "
          f"с ростом eta растёт амплитуда колебаний Es (скачок вверх между эпохами растёт с ~1.1x до ~2.8x), "
          f"хотя eta=0.2-0.3 сходится быстрее. eta=0.1 — середина диапазона: ~130 эпох при умеренных
колебаниях.")

def recursive_forecast(model, sn, p, start_k, steps):
    window = list(sn[start_k - p:start_k])
    preds = []
    for _ in range(steps):
        inp = np.array(window[::-1])
        y = model.predict(inp)
        preds.append(y)
        window = window[1:] + [y]
    return np.array(preds)

def predict_window(model, window_chrono, x_min, x_max):
    w = np.asarray(window_chrono, dtype=float)
    wn = normalize(w, x_min, x_max)
    y_norm = model.predict(wn[::-1])
    y_orig = denormalize(y_norm, x_min, x_max)
    return wn, y_norm, y_orig

def show_prediction(model, window_chrono, x_min, x_max, label, true_value=None, k=None):
    wn, y_norm, y_orig = predict_window(model, window_chrono, x_min, x_max)
    head = f"[{label}]" + (f" k={k}" if k is not None else "")
    print(head)
```

```
    print("  окно (исх. шкала):   ", np.array2string(np.asarray(window_chrono, dtype=float),
                                                     precision=4, separator=", "))
    print("  нормализованные входы:", np.array2string(wn, precision=4, separator=", "))
    out_of_range = np.any((wn < 0) | (wn > 1))
    if out_of_range:
        print("  (!) часть входов вне [0; 1] — значения вышли за диапазон обучающего участка")
    print(f"  прогноз (норм. шкала): {y_norm:.4f}")
    print(f"  прогноз (исх. шкала):  {y_orig:.4f}")
    if true_value is not None:
        print(f"  истинное x(k) = {true_value:.4f},  |ошибка| = {abs(true_value - y_orig):.4f}")
    print()

def show_from_series(model, series, p, x_min, x_max, k, label):
    show_prediction(model, series[k - p:k], x_min, x_max, label, true_value=series[k], k=k)

def demo_mode(model, series, p, x_min, x_max):
    print("\n" + "=" * 78)
    print("РЕЖИМ ФУНКЦИОНИРОВАНИЯ: примеры работы обученной сети (p=%d)" % p)
    print("=" * 78)
    print("--- 3 окна из обучающего участка ---")
    for k in (20, 70, 120):
        show_from_series(model, series, p, x_min, x_max, k, "train")
    print("--- 3 окна из тестового участка ---")
    for k in (145, 170, 195):
        show_from_series(model, series, p, x_min, x_max, k, "test")
    print("--- пользовательские окна ---")
    custom = list(series[150 - p:150])
    custom[p // 2] += 3.0
    show_prediction(model, custom, x_min, x_max,
                    "пользовательское окно с выбросом (+3.0 в середине окна)",
                    true_value=series[150], k=150)
    show_prediction(model, [0.5, 0.9, 1.3, 1.1, 0.6, 0.1][:p] if p == 6 else list(series[:p]),
                    x_min, x_max, "пользовательское окно (произвольные значения)")

def interactive_mode(model, series, p, x_min, x_max):
    print("\n--- Интерактивный режим ---")
    print(f"Команды: 'k <номер>'  — взять окно из ряда (p <= k <= {len(series) - 1});")
    print(f"         'v v1 v2 ... v{p}' — ввести {p} значений в исходной шкале "
          f"в хронологическом порядке (старое -> новое);")
    print("         'q' — выход.")
    while True:
        raw = input(">>> ").strip()
        if raw.lower() in ("q", "quit", "exit", ""):
            print("Выход из интерактивного режима.")
            break
        parts = raw.replace(",", " ").split()
        try:
            if parts[0].lower() == "k":
                k = int(parts[1])
                if not (p <= k < len(series)):
                    print(f"k должно быть в диапазоне [{p}; {len(series) - 1}].")
                    continue
                show_from_series(model, series, p, x_min, x_max, k, "из ряда")
            elif parts[0].lower() == "v":
                vals = [float(t) for t in parts[1:]]
                if len(vals) != p:
                    print(f"Нужно ровно {p} значений, получено {len(vals)}.")
                    continue
                show_prediction(model, vals, x_min, x_max, "пользовательское окно")
            else:
                print("Неизвестная команда. Примеры: 'k 150' или 'v 0.1 0.5 0.9 1.2 1.0 0.7'")
        except (ValueError, IndexError):
            print("Некорректный ввод. Примеры: 'k 150' или 'v 0.1 0.5 0.9 1.2 1.0 0.7'")

def manual_step_check(V=11, eta=0.1):
    X = np.array([0.2, 0.5, 0.8])
    d = 0.6
    W1 = np.array([[0.05 * V, -0.2],
                   [0.3, 0.05 * V],
                   [-0.1, 0.4]])
    b1 = np.array([0.1, -0.1])
    W2 = np.array([0.5, -0.2 * V])
    b2 = 0.2

    s = X @ W1 + b1
    h = sigmoid(s)
    y = float(h @ W2 + b2)
    e = d - y
    delta_h = e * W2 * h * (1 - h)
    W2_new = W2 + eta * e * h
    b2_new = b2 + eta * e
    W1_new = W1 + eta * np.outer(X, delta_h)
    b1_new = b1 + eta * delta_h
```

```
    np.set_printoptions(precision=6, suppress=True)
    print("\n" + "=" * 78)
    print(f"ПРОВЕРКА РУЧНОГО РАСЧЁТА (сеть 3-2-1, V={V}, eta={eta})")
    print("=" * 78)
    print("Forward:")
    print("  s (вход скрытых)  =", s)
    print("  h = sigma(s)      =", h)
    print(f"  y (линейный выход)= {y:.6f}")
    print(f"  ошибка e = d - y  = {e:.6f}   (E = 1/2 e^2 = {0.5 * e * e:.6f})")
    print("Backward:")
    print(f"  delta_out         = e = {e:.6f}")
    print("  delta_hidden      =", delta_h, "  (= e * W_out * h * (1-h))")
    print("Новые веса (eta = %s):" % eta)
    print("  W_out_new         =", W2_new, f"   b_out_new = {b2_new:.6f}")
    print("  W1_new (строки — входы, столбцы — скрытые нейроны):")
    print(W1_new)
    print("  b1_new            =", b1_new)

def plot_series(series, filename="lab4_series.png"):
    k = np.arange(len(series))
    plt.figure(figsize=(9, 4.5))
    plt.plot(k, series, color="tab:blue", label="x(k)")
    plt.axvline(TRAIN_END - 0.5, color="k", linestyle="--", label="граница train/test")
    plt.axvspan(TRAIN_END - 0.5, len(series), color="orange", alpha=0.12)
    plt.text(TRAIN_END / 2, series.max() * 0.95, "обучающий участок", ha="center")
    plt.text((TRAIN_END + len(series)) / 2, series.max() * 0.95, "тестовый участок", ha="center")
    plt.title("Временной ряд x(k) = a·cos(b·k) + c·sin(d·k), вариант 11")
    plt.xlabel("k"); plt.ylabel("x(k)")
    plt.legend(loc="lower left"); plt.grid(alpha=0.3)
    plt.tight_layout(); plt.savefig(filename, dpi=150); plt.close()
    print(f"Сохранён график: {filename}")

def plot_series_forecast(base, series, filename="lab4_series_forecast.png"):
    ds = base["ds"]
    k = np.arange(len(series))
    plt.figure(figsize=(10, 4.8))
    plt.plot(k, series, color="tab:blue", linewidth=1.5, label="эталон x(k)")
    plt.plot(ds.Ktr, base["pred_tr"], "--", color="tab:green", linewidth=1.2,
             label="одношаговый прогноз (train)")
    plt.plot(ds.Kte, base["pred_te"], "--", color="tab:red", linewidth=1.4,
             label="одношаговый прогноз (test)")
    plt.axvline(TRAIN_END - 0.5, color="k", linestyle=":", label="граница train/test")
    plt.title(f"Одношаговый прогноз сети {base['p']}-{base['N']}-1 на обучающем и тестовом участках")
    plt.xlabel("k"); plt.ylabel("x(k)")
    plt.legend(loc="upper center", bbox_to_anchor=(0.5, -0.14), ncol=4, fontsize=8)
    plt.grid(alpha=0.3)
    plt.tight_layout(); plt.savefig(filename, dpi=150, bbox_inches="tight"); plt.close()
    print(f"Сохранён график: {filename}")

def plot_convergence(curves, filename="lab4_convergence.png"):
    plt.figure(figsize=(8, 5))
    for label, hist in curves:
        plt.plot(range(1, len(hist) + 1), hist, label=label)
    plt.axhline(EE, color="gray", linestyle="--", linewidth=0.9, label=f"Ee = {EE:g}")
    plt.yscale("log")
    plt.title("Сходимость: средняя ошибка Es по эпохам\n(лог. шкала по Y; по X — тоже лог., т.к. p=2 идёт 20 000
эпох)")
    plt.xlabel("Эпоха"); plt.ylabel("Es (норм. шкала)")
    plt.xscale("log")
    plt.legend(); plt.grid(alpha=0.3, which="both")
    plt.tight_layout(); plt.savefig(filename, dpi=150); plt.close()
    print(f"Сохранён график: {filename}")

def plot_scatter(base, filename="lab4_scatter.png"):
    y, yh = base["y_te"], base["pred_te"]
    lo, hi = min(y.min(), yh.min()), max(y.max(), yh.max())
    plt.figure(figsize=(5.5, 5.5))
    plt.scatter(y, yh, s=22, color="tab:red", alpha=0.8, label="тестовые точки")
    plt.plot([lo, hi], [lo, hi], "k--", label="y = x")
    plt.title("Тестовый участок: эталон - прогноз")
    plt.xlabel("эталон"); plt.ylabel("прогноз сети")
    plt.legend(); plt.grid(alpha=0.3); plt.axis("equal")
    plt.tight_layout(); plt.savefig(filename, dpi=150); plt.close()
    print(f"Сохранён график: {filename}")

def plot_recursive(ks, ref, rec, errs, thr, step_exceed, label, filename="lab4_recursive.png"):
    fig, axes = plt.subplots(1, 2, figsize=(12, 4.6))
    axes[0].plot(ks, ref, "o-", color="tab:blue", markersize=3, label="эталон")
    axes[0].plot(ks, rec, "s--", color="tab:red", markersize=3, label="рекурсивный прогноз")
    axes[0].set_title(f"Рекурсивный прогноз на {len(ks)} шагов ({label})")
    axes[0].set_xlabel("k"); axes[0].set_ylabel("x(k)")
    axes[0].legend(loc="upper center", bbox_to_anchor=(0.5, -0.14), ncol=2); axes[0].grid(alpha=0.3)

    steps = np.arange(1, len(errs) + 1)
```

```
    axes[1].plot(steps, errs, "o-", color="tab:purple", markersize=3, label="|ошибка|")
    axes[1].axhline(thr, color="gray", linestyle="--", label=f"5 % размаха = {thr:.3f}")
    if step_exceed is not None:
        axes[1].axvline(step_exceed, color="red", linestyle=":", label=f"превышение на шаге {step_exceed}")
    axes[1].set_ylim(0, max(thr, errs.max()) * 1.45)
    axes[1].set_title("Рост абсолютной ошибки по шагам")
    axes[1].set_xlabel("номер шага"); axes[1].set_ylabel("|x - x^|")
    axes[1].legend(loc="upper center", bbox_to_anchor=(0.5, -0.14), ncol=2, fontsize=9); axes[1].grid(alpha=0.3)
    plt.tight_layout(); plt.savefig(filename, dpi=150, bbox_inches="tight"); plt.close()
    print(f"Сохранён график: {filename}")

def plot_studies(p_vals, p_res, n_vals, n_res, filename="lab4_studies.png"):
    fig, axes = plt.subplots(1, 2, figsize=(11, 4.4))
    for ax, vals, res, xl in ((axes[0], p_vals, p_res, "размер окна p"),
                              (axes[1], n_vals, n_res, "число скрытых нейронов N")):
        m = [summarize(r)["te_mean"] for r in res]
        s = [summarize(r)["te_std"] for r in res]
        tr = [summarize(r)["tr_mean"] for r in res]
        ax.errorbar(range(len(vals)), m, yerr=s, fmt="o-", capsize=4, color="tab:red", label="test MAE
(ср.±ст.)")
        ax.plot(range(len(vals)), tr, "s--", color="tab:green", label="train MAE (ср.)")
        ax.set_xticks(range(len(vals))); ax.set_xticklabels(vals)
        if ax is axes[0]:
            ax.set_yscale("log")
        else:
            ax.set_ylim(0, max(max(m), max(tr)) * 1.6)
        ax.set_xlabel(xl); ax.set_ylabel("MAE (исходная шкала)")
        ax.grid(alpha=0.3, which="both"); ax.legend()
    axes[0].set_title("Влияние размера окна p (N = %d)" % N_VAR)
    axes[1].set_title("Влияние числа нейронов N (p = %d)" % P_VAR)
    plt.tight_layout(); plt.savefig(filename, dpi=150); plt.close()
    print(f"Сохранён график: {filename}")

def noise_experiment(series, N_values=(N_VAR, 4 * N_VAR), p=P_VAR, seeds=SEEDS):
    print("\n" + "=" * 78)
    print("ЭКСПЕРИМЕНТ С ШУМОМ (обучение на зашумлённом train, тест на ЧИСТОМ ряде)")
    print("=" * 78)
    rng = np.random.default_rng(12345)
    x_min0, x_max0 = fit_minmax(series[:TRAIN_END])
    sigma = NOISE_STD_FRAC * (x_max0 - x_min0)
    noisy = series.copy()
    noisy[:TRAIN_END] += rng.normal(0.0, sigma, size=TRAIN_END)
    print(f"Гауссов шум: sigma = {sigma:.4f} ({NOISE_STD_FRAC * 100:.0f} % размаха train), "
          f"p={p}; для сопоставимости обе серии обучаются фиксированные {NOISE_EPOCHS} эпох (Ее отключён).")
    print(f"{'обучение':>10} | {'N':>3} | {'train MAE (свои цели)':>21} | "
          f"{'train MAE (чистый ряд)':>22} | {'test MAE (чистый)':>17}")
    table = {}
    for with_noise in (False, True):
        ser_train = noisy if with_noise else series
        x_min, x_max = fit_minmax(ser_train[:TRAIN_END])
        sn_tr = normalize(ser_train, x_min, x_max)
        sn_clean = normalize(series, x_min, x_max)
        Xtr, Ytr, _ = make_windows(sn_tr, p, p, TRAIN_END)
        Xtr_c, Ytr_c, _ = make_windows(sn_clean, p, p, TRAIN_END)
        Xte, Yte, _ = make_windows(sn_clean, p, TRAIN_END, len(series))
        for N in N_values:
            own, clean_tr, te = [], [], []
            for sd in seeds:
                m = MLP(p, N, seed=sd)
                m.train(Xtr, Ytr, ee=0.0, max_epochs=NOISE_EPOCHS)
                dn = lambda v: denormalize(v, x_min, x_max)
                own.append(mae(dn(Ytr), dn(m.predict_batch(Xtr))))
                clean_tr.append(mae(dn(Ytr_c), dn(m.predict_batch(Xtr_c))))
                te.append(mae(dn(Yte), dn(m.predict_batch(Xte))))
            table[(with_noise, N)] = (np.mean(own), np.mean(clean_tr), np.mean(te))
            tag = "с шумом" if with_noise else "без шума"
            print(f"{tag:>10} | {N:>3} | {np.mean(own):>21.4f} | {np.mean(clean_tr):>22.4f} |
{np.mean(te):>17.4f}",
                  flush=True)
    for N in N_values:
        r = table[(True, N)][2] / table[(False, N)][2]
        print(f"N={N}: шум в обучающих данных увеличил test MAE в {r:.2f} раза "
              f"(gap test - train по чистому ряду при шуме: {table[(True, N)][2] - table[(True, N)][1]:+.4f})")

def main():
    series = generate_series()
    print(f"Ряд сгенерирован: {len(series)} точек, min={series.min():.4f}, max={series.max():.4f}, "
          f"размах={series.max() - series.min():.4f}")
    crosses_zero = bool(np.any(np.sign(series[:-1]) * np.sign(series[1:]) < 0))
    print(f"Ряд пересекает ноль: {'да -> вместо MAPE используем sMAPE' if crosses_zero else 'нет'}")
    plot_series(series)

    x_min, x_max = fit_minmax(series[:TRAIN_END])
    sn = normalize(series, x_min, x_max)
```

```
test_norm = sn[TRAIN_END:]
n_out = int(np.sum((test_norm < 0) | (test_norm > 1)))
print(f"\nНормализация по train: x_min={x_min:.4f}, x_max={x_max:.4f}")
print("  x_norm = (x - x_min)/(x_max - x_min);   x = x_norm*(x_max - x_min) + x_min")
print(f"  тестовые значения в норм. шкале: [{test_norm.min():.4f}; {test_norm.max():.4f}], "
      f"вне [0;1]: {n_out} из {len(test_norm)}")

ds_var = Dataset(series, P_VAR)
print(f"\nСкользящее окно p={P_VAR}: обучающих примеров = {len(ds_var.Xtr)} "
      f"(цели k={P_VAR}..{TRAIN_END - 1}), тестовых = {len(ds_var.Xte)} "
      f"(цели k={TRAIN_END}..{len(series) - 1})")

lr_sweep(series)

print("\n" + "=" * 78)
print(f"БАЗОВЫЙ ЭКСПЕРИМЕНТ: p={P_VAR}, N={N_VAR}, eta={LEARNING_RATE}, seed={BASE_SEED}")
print("=" * 78)
base = run_config(series, P_VAR, N_VAR, BASE_SEED)
print(f"Эпох: {base['epochs']}  (критерий Es<=Ee достигнут: {base['converged']})")
print(f"Итоговая Es = {base['Es']:.3e}  (норм. шкала)")
print(f"{'':<8}{'MAE':>10}{'RMSE':>10}{'sMAPE,%':>10}")
for name in ("train", "test"):
    m = base[name]
    print(f"{name:<8}{m['MAE']:>10.4f}{m['RMSE']:>10.4f}{m['sMAPE']:>10.3f}")
mp_te = mape(base["y_te"], base["pred_te"])
rel = np.abs(base["y_te"] - base["pred_te"]) / (np.abs(base["y_te"]) + EPS) * 100.0
j = int(np.argmax(rel))
print(f"(для сравнения, MAPE на test = {mp_te:.1f} %, но отдельная точка может дать {rel[j]:.0f} % "
      f"при |x| = {abs(base['y_te'][j]):.4f} (≈0) — поэтому MAPE нестабильна, используем sMAPE)")

print("\n" + "=" * 78)
print(f"ИССЛЕДОВАНИЕ ОКНА (N={N_VAR}), p in {P_LIST}")
print("=" * 78)
p_results = []
for p in P_LIST:
    print(f"  --- p = {p} ---", flush=True)
    p_results.append(run_series(series, p, N_VAR))
print_study_table(f"Таблица: влияние окна p (N={N_VAR}, 5 запусков)", "p", P_LIST, p_results)

print("\n" + "=" * 78)
print(f"ИССЛЕДОВАНИЕ СКРЫТОГО СЛОЯ (p={P_VAR}), N in {N_LIST}")
print("=" * 78)
n_results = []
for N in N_LIST:
    print(f"  --- N = {N} ---", flush=True)
    n_results.append(run_series(series, P_VAR, N))
print_study_table(f"Таблица: влияние числа нейронов N (p={P_VAR}, 5 запусков)", "N", N_LIST, n_results)

print()
plot_series_forecast(base, series)
curves = [(f"базовая p={P_VAR}, N={N_VAR}", base["history"]),
          (f"p=2, N={N_VAR}", p_results[0][0]["history"]),
          (f"p={2 * P_VAR}, N={N_VAR}", p_results[3][0]["history"])]
plot_convergence(curves)
plot_scatter(base)
plot_studies(P_LIST, p_results, N_LIST, n_results)

candidates = {}
for res in p_results + n_results:
    key = (res[0]["p"], res[0]["N"])
    candidates[key] = res
best_key = min(candidates, key=lambda kk: summarize(candidates[kk])["te_mean"])
best_res = candidates[best_key]
best_run = min(best_res, key=lambda r: r["test"]["MAE"])
bp, bN = best_key
print("\n" + "=" * 78)
print(f"МНОГОШАГОВЫЙ ПРОГНОЗ: лучшая по среднему test MAE конфигурация p={bp}, N={bN} "
      f"(средний test MAE = {summarize(best_res)['te_mean']:.4f}; берём запуск seed={best_run['seed']})")
print("=" * 78)
ds_b = best_run["ds"]
pred_n = recursive_forecast(best_run["model"], ds_b.sn, bp, TRAIN_END, FORECAST_STEPS)
rec = ds_b.denorm(pred_n)
ks = np.arange(TRAIN_END, TRAIN_END + FORECAST_STEPS)
ref = series[ks]
errs = np.abs(ref - rec)
thr = ERR_THRESHOLD_FRAC * (series.max() - series.min())
exceed = np.where(errs > thr)[0]
step_exceed = int(exceed[0]) + 1 if len(exceed) else None
one_step_err = np.abs(best_run["pred_te"][:FORECAST_STEPS] - ref)
print(f"Порог 5 % размаха ряда = {thr:.4f}")
print(f"Одношаговый прогноз на тех же {FORECAST_STEPS} точках: MAE = {one_step_err.mean():.4f}, "
      f"макс. |ошибка| = {one_step_err.max():.4f}")
print(f"Рекурсивный прогноз: MAE = {errs.mean():.4f}, макс. |ошибка| = {errs.max():.4f}")
if step_exceed is None:
```

```
        print(f"Ошибка НЕ превышает 5 % размаха ни на одном из {FORECAST_STEPS} шагов.")
    else:
        print(f"Ошибка впервые превышает 5 % размаха на шаге {step_exceed} (k={TRAIN_END + step_exceed - 1}).")
    print("Ошибка по шагам:", np.array2string(errs, precision=4, separator=", ", max_line_width=100))
    plot_recursive(ks, ref, rec, errs, thr, step_exceed, f"p={bp}, N={bN}")

    n_full = len(series) - TRAIN_END
    rec_full = ds_b.denorm(recursive_forecast(best_run["model"], ds_b.sn, bp, TRAIN_END, n_full))
    errs_full = np.abs(series[TRAIN_END:] - rec_full)
    ex_full = np.where(errs_full > thr)[0]
    msg = (f"впервые превышает 5 %% размаха на шаге %d" % (int(ex_full[0]) + 1)) if len(ex_full) \
        else "не превышает 5 % размаха"

    noise_experiment(series)

    manual_step_check(V=11, eta=0.1)

    demo_mode(base["model"], series, P_VAR, base["ds"].x_min, base["ds"].x_max)

    choice = input("Запустить интерактивный режим (базовая сеть p=%d, N=%d)? (y/Enter — пропустить): "
                   % (P_VAR, N_VAR)).strip().lower()
    if choice == "y":
        interactive_mode(base["model"], series, P_VAR, base["ds"].x_min, base["ds"].x_max)
    else:
        print("Интерактивный режим пропущен.")

if __name__ == "__main__":
    main()
```

Результат работы программы

```
Ряд сгенерирован: 200 точек, min=-2.0914, max=2.0828, размах=4.1742
Ряд пересекает ноль: да -> вместо MAPE используем sMAPE
Сохранён график: lab4_series.png

Нормализация по train: x_min=-2.0914, x_max=2.0828
  x_norm = (x - x_min)/(x_max - x_min);   x = x_norm*(x_max - x_min) + x_min
  тестовые значения в норм. шкале: [0.0056; 0.9846], вне [0;1]: 0 из 60

Скользящее окно p=6: обучающих примеров = 134 (цели k=6..139), тестовых = 60 (цели k=140..199)

Подбор скорости обучения (базовая конфигурация p=6, N=5, 5 запусков):
«Макс. скачок Es» = max по эпохам Es(t)/Es(t-1): мера амплитуды колебаний в онлайн-режиме.
   eta | эпох (ср.) |  сошлось | test MAE (ср.) | макс. скачок Es
  0.01 |     1257.4 |    5/5   |         0.0351 |           1.09x
  0.03 |      421.0 |    5/5   |         0.0349 |           1.37x
  0.05 |      254.6 |    5/5   |         0.0348 |           1.60x
   0.1 |      129.8 |    5/5   |         0.0339 |           1.96x
   0.2 |       70.0 |    5/5   |         0.0322 |           2.35x
   0.3 |       49.6 |    5/5   |         0.0320 |           2.77x
Выбрано eta = 0.1.

================================================================================
БАЗОВЫЙ ЭКСПЕРИМЕНТ: p=6, N=5, eta=0.1, seed=1
================================================================================
Эпох: 135  (критерий Es<=Ee достигнут: True)
Итоговая Es = 9.934e-05  (норм. шкала)
               MAE      RMSE   sMAPE,%
train       0.0348    0.0416     9.985
test        0.0339    0.0407     9.764
(для сравнения, MAPE на test = 8.8 %, но отдельная точка может дать 95 % при |x| = 0.0584 (≈0) — поэтому MAPE
нестабильна, используем sMAPE)

================================================================================
ИССЛЕДОВАНИЕ ОКНА (N=5), p in [2, 4, 6, 12]
================================================================================
  --- p = 2 ---
    p= 2 N= 5 seed=1: эпох=20000 Es=3.149e-03 сошлось=False testMAE=0.1946
    p= 2 N= 5 seed=2: эпох=20000 Es=2.608e-03 сошлось=False testMAE=0.1786
    p= 2 N= 5 seed=3: эпох=20000 Es=2.886e-03 сошлось=False testMAE=0.1876
    p= 2 N= 5 seed=4: эпох=20000 Es=2.768e-03 сошлось=False testMAE=0.1819
    p= 2 N= 5 seed=5: эпох=20000 Es=2.855e-03 сошлось=False testMAE=0.1877
  --- p = 4 ---
    p= 4 N= 5 seed=1: эпох= 1376 Es=9.860e-05 сошлось=True testMAE=0.0328
    p= 4 N= 5 seed=2: эпох= 1376 Es=9.986e-05 сошлось=True testMAE=0.0337
    p= 4 N= 5 seed=3: эпох= 1339 Es=9.858e-05 сошлось=True testMAE=0.0347
    p= 4 N= 5 seed=4: эпох= 1334 Es=9.767e-05 сошлось=True testMAE=0.0342
    p= 4 N= 5 seed=5: эпох= 1329 Es=9.889e-05 сошлось=True testMAE=0.0338
  --- p = 6 ---
    p= 6 N= 5 seed=1: эпох=  135 Es=9.934e-05 сошлось=True testMAE=0.0339
    p= 6 N= 5 seed=2: эпох=  130 Es=9.212e-05 сошлось=True testMAE=0.0339
    p= 6 N= 5 seed=3: эпох=  133 Es=9.344e-05 сошлось=True testMAE=0.0332
    p= 6 N= 5 seed=4: эпох=  125 Es=9.787e-05 сошлось=True testMAE=0.0356
    p= 6 N= 5 seed=5: эпох=  126 Es=8.688e-05 сошлось=True testMAE=0.0330
  --- p = 12 ---
    p=12 N= 5 seed=1: эпох=   23 Es=9.423e-05 сошлось=True testMAE=0.0280
    p=12 N= 5 seed=2: эпох=   46 Es=9.810e-05 сошлось=True testMAE=0.0347
    p=12 N= 5 seed=3: эпох=   21 Es=8.076e-05 сошлось=True testMAE=0.0295
```

```
    p=12 N= 5 seed=4: эпох=   40 Es=9.880e-05 сошлось=True testMAE=0.0323
    p=12 N= 5 seed=5: эпох=   19 Es=8.502e-05 сошлось=True testMAE=0.0303

Таблица: влияние окна p (N=5, 5 запусков)
   p |    эпохи (ср.±ст.) |     [min; max] | train MAE |   test MAE (ср.±ст.) |         [min; max] | не сошлось
-----------------------------------------------------------------------------------------------------------------
   2 |    20000.0±0.0     | [20000; 20000] |    0.1850 |      0.1861±0.0061   | [0.1786;0.1946] |    5/5
   4 |     1350.8±23.3    | [ 1329;  1376] |    0.0334 |      0.0338±0.0007   | [0.0328;0.0347] |    0/5
   6 |      129.8±4.3     | [  125;   135] |    0.0343 |      0.0339±0.0010   | [0.0330;0.0356] |    0/5
  12 |       29.8±12.3    | [   19;    46] |    0.0314 |      0.0310±0.0026   | [0.0280;0.0347] |    0/5

============================================================================
ИССЛЕДОВАНИЕ СКРЫТОГО СЛОЯ (p=6), N in [1, 2, 5, 10]
============================================================================
  --- N = 1 ---
    p= 6 N= 1 seed=1: эпох=  154 Es=9.074e-05 сошлось=True testMAE=0.0341
    p= 6 N= 1 seed=2: эпох=  142 Es=9.525e-05 сошлось=True testMAE=0.0342
    p= 6 N= 1 seed=3: эпох=  142 Es=9.305e-05 сошлось=True testMAE=0.0350
    p= 6 N= 1 seed=4: эпох=  155 Es=9.873e-05 сошлось=True testMAE=0.0349
    p= 6 N= 1 seed=5: эпох=  163 Es=9.702e-05 сошлось=True testMAE=0.0353
  --- N = 2 ---
    p= 6 N= 2 seed=1: эпох=  152 Es=9.803e-05 сошлось=True testMAE=0.0348
    p= 6 N= 2 seed=2: эпох=  166 Es=9.647e-05 сошлось=True testMAE=0.0346
    p= 6 N= 2 seed=3: эпох=  160 Es=9.364e-05 сошлось=True testMAE=0.0344
    p= 6 N= 2 seed=4: эпох=  144 Es=9.329e-05 сошлось=True testMAE=0.0330
    p= 6 N= 2 seed=5: эпох=  146 Es=9.942e-05 сошлось=True testMAE=0.0355
  --- N = 5 ---
    p= 6 N= 5 seed=1: эпох=  135 Es=9.934e-05 сошлось=True testMAE=0.0339
    p= 6 N= 5 seed=2: эпох=  130 Es=9.212e-05 сошлось=True testMAE=0.0339
    p= 6 N= 5 seed=3: эпох=  133 Es=9.344e-05 сошлось=True testMAE=0.0332
    p= 6 N= 5 seed=4: эпох=  125 Es=9.787e-05 сошлось=True testMAE=0.0356
    p= 6 N= 5 seed=5: эпох=  126 Es=8.688e-05 сошлось=True testMAE=0.0330
  --- N = 10 ---
    p= 6 N=10 seed=1: эпох=  136 Es=9.146e-05 сошлось=True testMAE=0.0333
    p= 6 N=10 seed=2: эпох=  116 Es=9.811e-05 сошлось=True testMAE=0.0332
    p= 6 N=10 seed=3: эпох=  108 Es=8.548e-05 сошлось=True testMAE=0.0319
    p= 6 N=10 seed=4: эпох=  137 Es=9.662e-05 сошлось=True testMAE=0.0349
    p= 6 N=10 seed=5: эпох=  128 Es=9.793e-05 сошлось=True testMAE=0.0371

Таблица: влияние числа нейронов N (p=6, 5 запусков)
   N |    эпохи (ср.±ст.) |     [min; max] | train MAE |   test MAE (ср.±ст.) |         [min; max] | не сошлось
-----------------------------------------------------------------------------------------------------------------
   1 |      151.2±9.1     | [  142;   163] |    0.0348 |      0.0347±0.0005   | [0.0341;0.0353] |    0/5
   2 |      153.6±9.3     | [  144;   166] |    0.0352 |      0.0345±0.0009   | [0.0330;0.0355] |    0/5
   5 |      129.8±4.3     | [  125;   135] |    0.0343 |      0.0339±0.0010   | [0.0330;0.0356] |    0/5
  10 |      125.0±12.7    | [  108;   137] |    0.0343 |      0.0341±0.0020   | [0.0319;0.0371] |    0/5

Сохранён график: lab4_series_forecast.png
Сохранён график: lab4_convergence.png
Сохранён график: lab4_scatter.png
Сохранён график: lab4_studies.png

============================================================================
МНОГОШАГОВЫЙ ПРОГНОЗ: лучшая по среднему test MAE конфигурация p=12, N=5 (средний test MAE = 0.0310; берём запуск seed=1)
============================================================================
Порог 5 % размаха ряда = 0.2087
Одношаговый прогноз на тех же 30 точках: MAE = 0.0213, макс. |ошибка| = 0.0661
Рекурсивный прогноз: MAE = 0.0533, макс. |ошибка| = 0.1222
Ошибка НЕ превышает 5 % размаха ни на одном из 30 шагов.
Ошибка по шагам: [0.0661, 0.0129, 0.0428, 0.075 , 0.0995, 0.0754, 0.0486, 0.0182, 0.0745, 0.0637, 0.0342, 0.0254,
 0.0211, 0.0122, 0.0394, 0.0465, 0.0037, 0.0617, 0.1222, 0.1162, 0.0624, 0.0321, 0.0057, 0.0488,
 0.0576, 0.0362, 0.059 , 0.0858, 0.0872, 0.066 ]
Сохранён график: lab4_recursive.png
[Дополнительно] прогноз на весь тест (60 шагов): макс. |ошибка| = 0.2520, ошибка впервые превышает 5 % размаха на шаге
52; MAE по шагам 1-30 = 0.0533, по шагам 31-60 = 0.1095

============================================================================
ЭКСПЕРИМЕНТ С ШУМОМ (обучение на зашумлённом train, тест на ЧИСТОМ ряде)
============================================================================
Гауссов шум: sigma = 0.0835 (2 % размаха train), p=6; для сопоставимости обе серии обучаются фиксированные 3000 эпох (Ee
отключён).
  обучение |   N | train MAE (свои цели) | train MAE (чистый ряд) | test MAE (чистый)
  без шума |   5 |                0.0051 |                 0.0051 |            0.0050
  без шума |  20 |                0.0039 |                 0.0039 |            0.0039
   с шумом |   5 |                0.1619 |                 0.0793 |            0.0779
   с шумом |  20 |                0.1628 |                 0.0776 |            0.0759
N=5: шум в обучающих данных увеличил test MAE в 15.43 раза (gap test - train по чистому ряду при шуме: -0.0015)
N=20: шум в обучающих данных увеличил test MAE в 19.67 раза (gap test - train по чистому ряду при шуме: -0.0016)

============================================================================
ПРОВЕРКА РУЧНОГО РАСЧЁТА (сеть 3-2-1, V=11, eta=0.1)
============================================================================
Forward:
  s (вход скрытых)  = [0.28  0.455]
  h = sigma(s)      = [0.569546 0.611827]
  y (линейный выход)= -0.861247
  ошибка e = d - y  = 1.461247   (E = 1/2 e^2 = 1.067622)
Backward:
  delta_out         = e = 1.461247
  delta_hidden      = [ 0.179122 -0.763484]   (= e * W_out * h * (1-h))
Новые веса (eta = 0.1):
```

```
  W_out_new         = [ 0.583225 -2.110597]    b_out_new = 0.346125
  W1_new (строки — входы, столбцы — скрытые нейроны):
[[ 0.553582 -0.21527 ]
 [ 0.308956  0.511826]
 [-0.08567   0.338921]]
  b1_new            = [ 0.117912 -0.176348]

================================================================================
РЕЖИМ ФУНКЦИОНИРОВАНИЯ: примеры работы обученной сети (p=6)
================================================================================
--- 3 окна из обучающего участка ---
[train] k=20
  окно (исх. шкала):    [0.3666, 1.2964, 1.9726, 2.0442, 1.4662, 0.5299]
  нормализованные входы: [0.5889, 0.8116, 0.9736, 0.9908, 0.8523, 0.628 ]
  прогноз (норм. шкала): 0.4263
  прогноз (исх. шкала):  -0.3119
  истинное x(k) = -0.3212,  |ошибка| = 0.0093

[train] k=70
  окно (исх. шкала):    [0.3762, 0.5967, 1.0944, 1.5991, 1.7288, 1.2547]
  нормализованные входы: [0.5912, 0.644 , 0.7632, 0.8841, 0.9152, 0.8016]
  прогноз (норм. шкала): 0.5862
  прогноз (исх. шкала):  0.3554
  истинное x(k) = 0.2771,  |ошибка| = 0.0783

[train] k=120
  окно (исх. шкала):    [1.4331, 1.0976, 0.7382, 0.6256, 0.7556, 0.8639]
  нормализованные входы: [0.8443, 0.764 , 0.6779, 0.6509, 0.6821, 0.708 ]
  прогноз (норм. шкала): 0.6617
  прогноз (исх. шкала):  0.6705
  истинное x(k) = 0.6307,  |ошибка| = 0.0398

--- 3 окна из тестового участка ---
[test] k=145
  окно (исх. шкала):    [-1.8255, -1.6301, -1.0624, -0.5134, -0.2554, -0.2718]
  нормализованные входы: [0.0637, 0.1105, 0.2465, 0.378 , 0.4398, 0.4359]
  прогноз (норм. шкала): 0.4367
  прогноз (исх. шкала):  -0.2684
  истинное x(k) = -0.2951,  |ошибка| = 0.0268

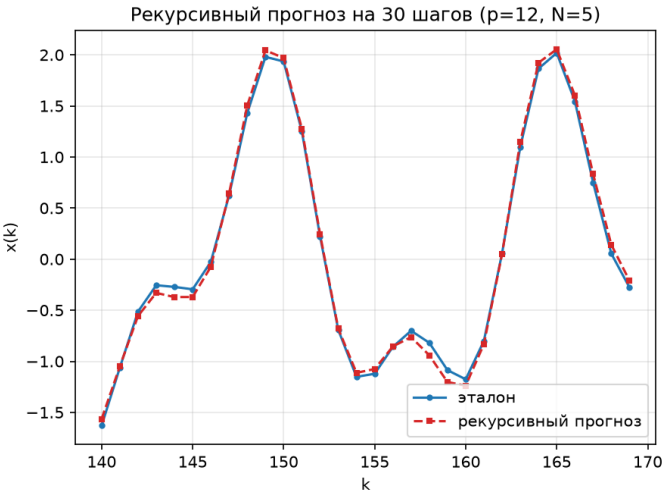
[test] k=170
  окно (исх. шкала):    [ 1.8651,  2.0184,  1.545 ,  0.7501,  0.0539, -0.2777]
  нормализованные входы: [0.9478, 0.9846, 0.8712, 0.6807, 0.5139, 0.4345]
  прогноз (норм. шкала): 0.4250
  прогноз (исх. шкала):  -0.3172
  истинное x(k) = -0.2838,  |ошибка| = 0.0334

[test] k=195
  окно (исх. шкала):    [-1.5597, -2.0681, -1.9213, -1.1812, -0.2116,  0.5495]
  нормализованные входы: [0.1274, 0.0056, 0.0408, 0.2181, 0.4503, 0.6327]
  прогноз (норм. шкала): 0.7180
  прогноз (исх. шкала):  0.9056
  истинное x(k) = 0.8563,  |ошибка| = 0.0493

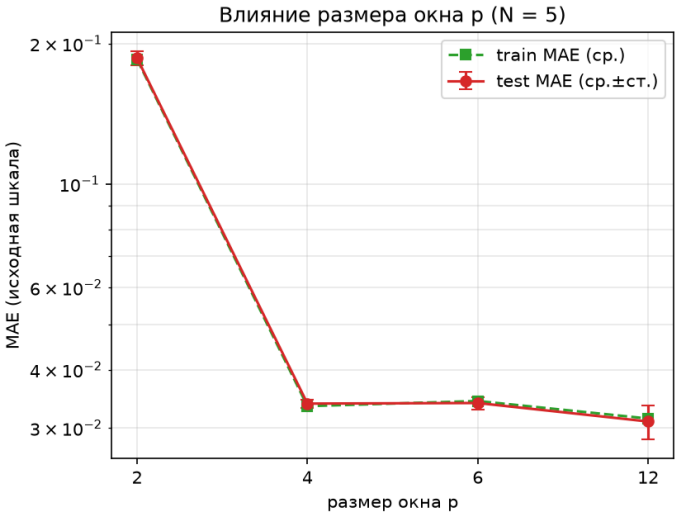
--- пользовательские окна ---
[пользовательское окно с выбросом (+3.0 в середине окна)] k=150
  окно (исх. шкала):    [-0.2718, -0.2951, -0.0262,  3.6227,  1.4289,  1.9802]
  нормализованные входы: [0.4359, 0.4303, 0.4948, 1.3689, 0.8434, 0.9754]
  (!) часть входов вне [0; 1] — значения вышли за диапазон обучающего участка
  прогноз (норм. шкала): 0.4762
  прогноз (исх. шкала):  -0.1039
  истинное x(k) = 1.9353,  |ошибка| = 2.0392

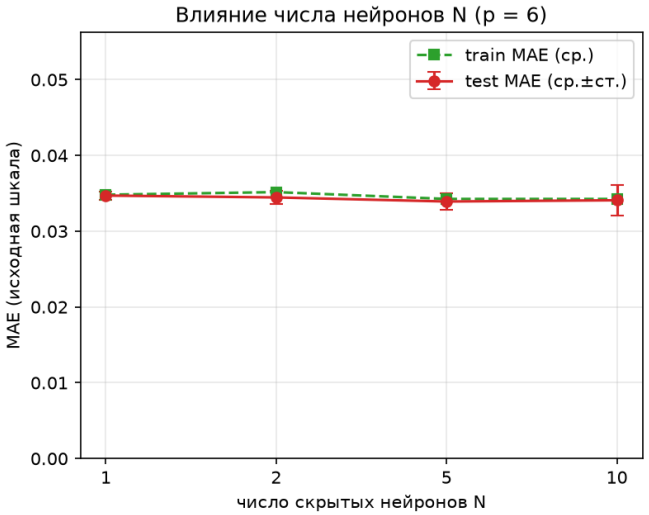
[пользовательское окно (произвольные значения)]
  окно (исх. шкала):    [0.5, 0.9, 1.3, 1.1, 0.6, 0.1]
  нормализованные входы: [0.6208, 0.7166, 0.8125, 0.7646, 0.6448, 0.525 ]
  прогноз (норм. шкала): 0.4165
  прогноз (исх. шкала):  -0.3528

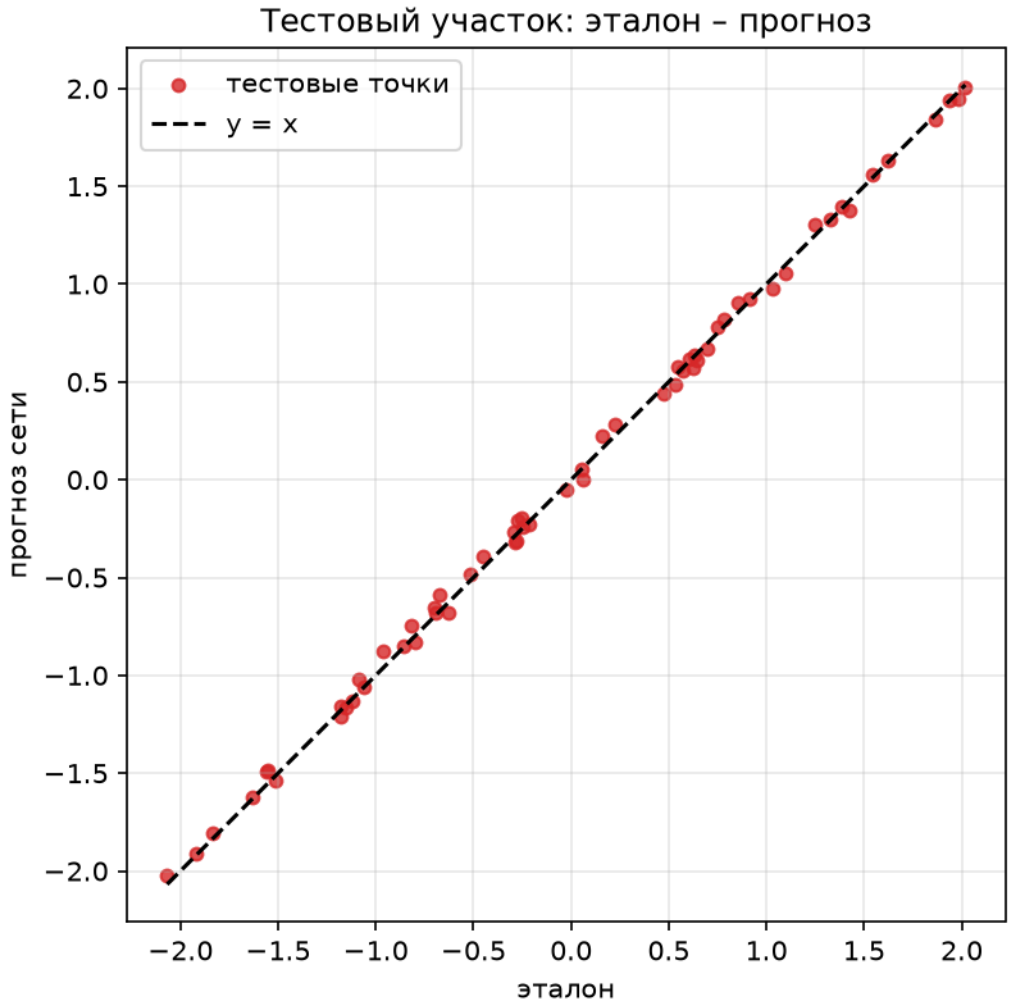
Запустить интерактивный режим (базовая сеть p=6, N=5)? (y/Enter — пропустить): Интерактивный режим пропущен.
```

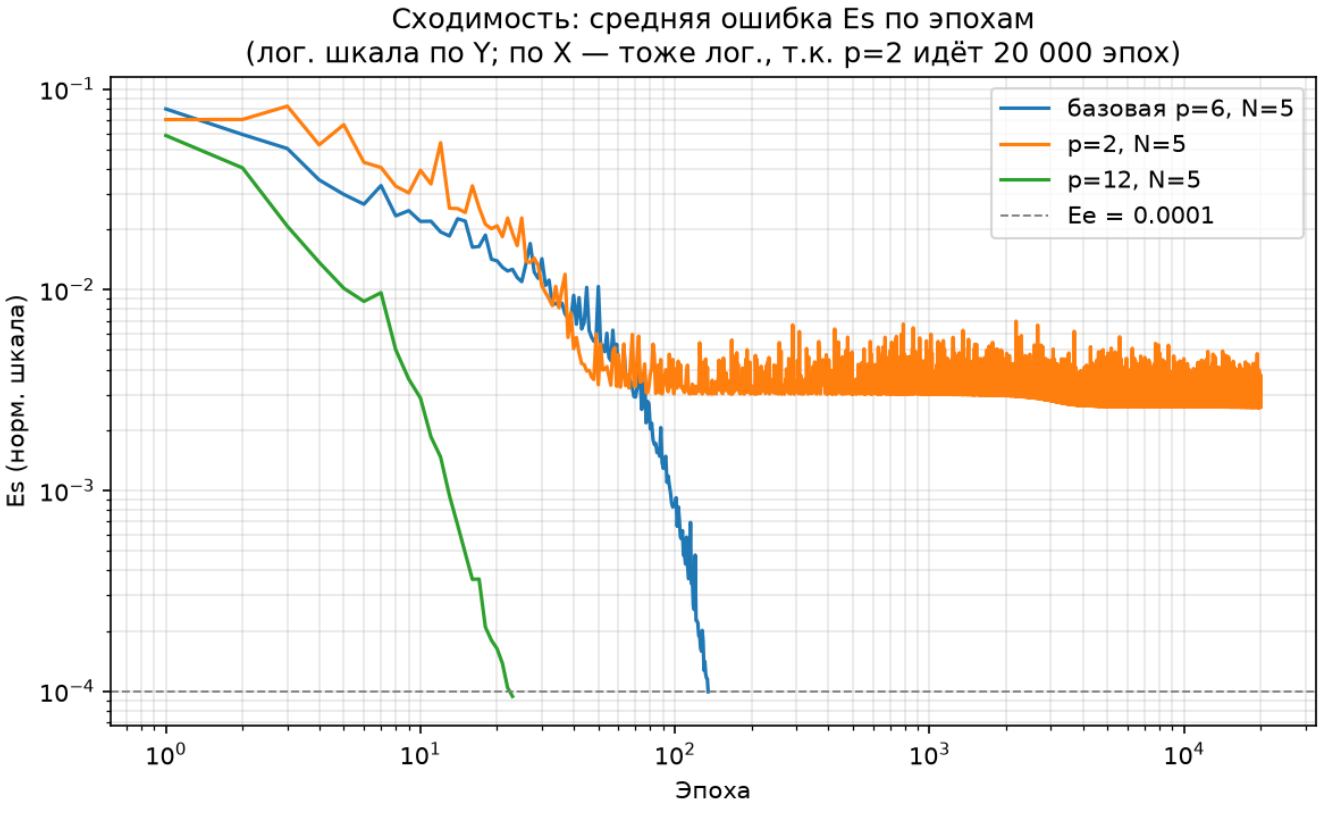

Рекурсивный прогноз на 30 шагов (p=12, N=5)
x(k)
k
эталон
рекурсивный прогноз

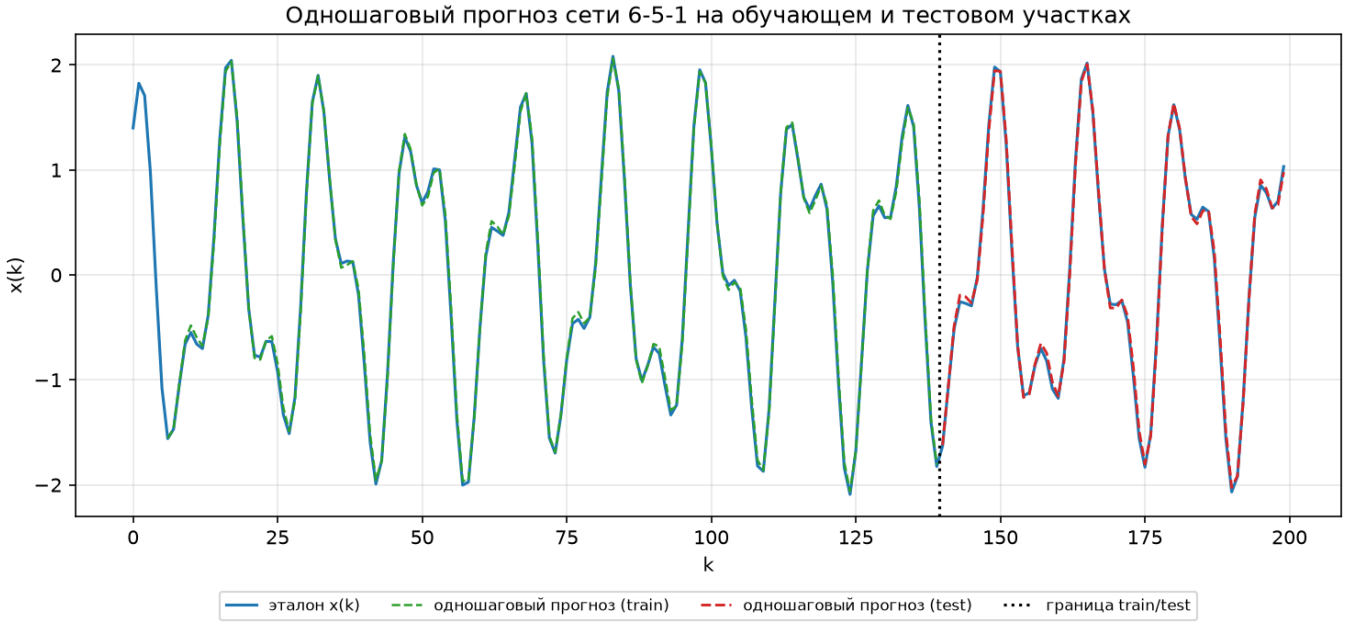

Рост абсолютной ошибки по шагам
|x - x^|
номер шага
|ошибка|
5 % размаха = 0.209

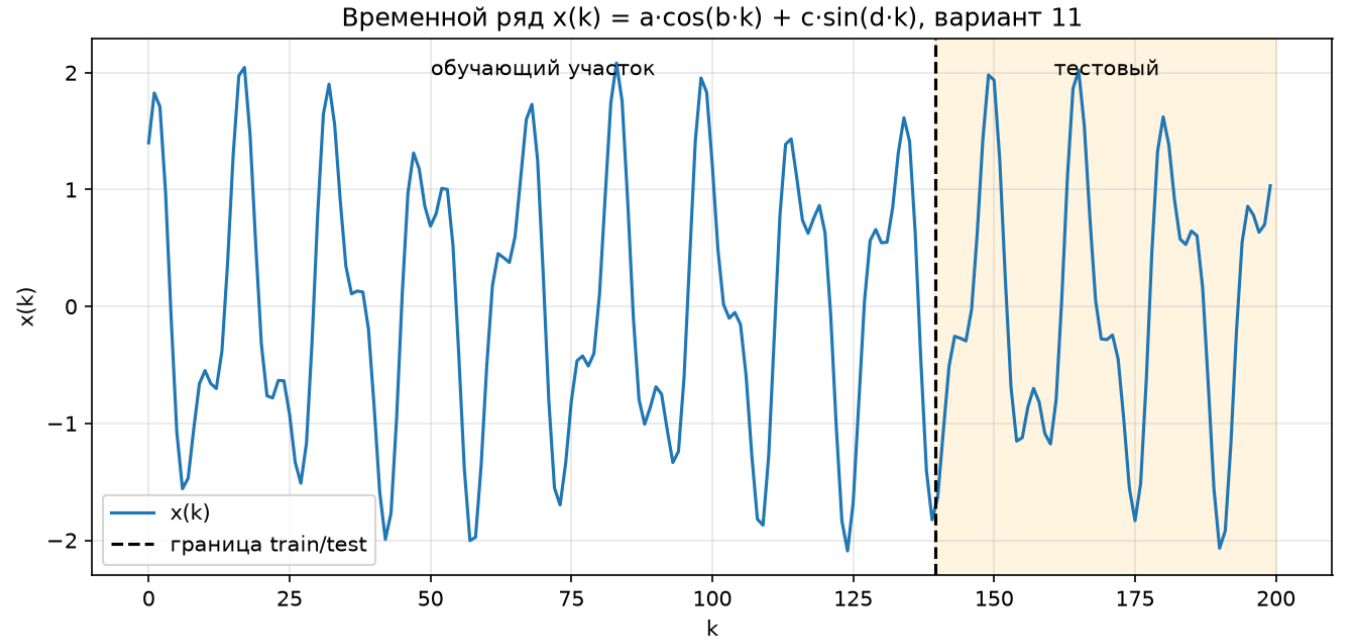

Влияние размера окна p (N = 5)
MAE (исходная шкала)
размер окна p
train MAE (ср.)
test MAE (ср.±ст.)


Влияние числа нейронов N (p = 6)
MAE (исходная шкала)
число скрытых нейронов N
train MAE (ср.)
test MAE (ср.±ст.)


Тестовый участок: эталон – прогноз
тестовые точки
y = x
прогноз сети
эталон
2.0
1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
−2.0
−2.0
−1.5
−1.0
−0.5
0.0
0.5
1.0
1.5
2.0


Сходимость: средняя ошибка Es по эпохам
(лог. шкала по Y; по X — тоже лог., т.к. p=2 идёт 20 000 эпох)
базовая p=6, N=5
p=2, N=5
p=12, N=5
Ee = 0.0001
Es (норм. шкала)
Эпоха
10^{-1}
10^{-2}
10^{-3}
10^{-4}
10^{0}
10^{1}
10^{2}
10^{3}
10^{4}





1. Как размер окна p влияет на качество прогноза? При каком p ошибка перестаёт уменьшаться? Почему p = 2 недостаточно, а слишком большое окно не даёт выигрыша?

При p = 2 критерий Ее не достигнут ни в одном запуске (Es ≈ 2.6–3.1·10⁻³, test MAE 0.186). При p = 4 test MAE = 0.0338 (1351 эпоха), при p = 6 – 0.0339 (130 эпох), при p = 12 – 0.0310 (30 эпох). Ошибка перестаёт уменьшаться с p = 4: сумма двух гармоник подчиняется линейной рекуррентности порядка 4, поэтому двух значений для определения обеих фаз мало, а более широкое окно новой информации не добавляет. Выигрыш при p = 12 сопоставим с разбросом (σ = 0.0026) и проявляется в скорости сходимости.

2. Как число нейронов N влияет на точность и скорость обучения? Наблюдается ли переобучение? Достаточно ли минимального N?

Точность почти не зависит от N: test MAE 0.0347 / 0.0345 / 0.0339 / 0.0341 для N = 1, 2, 5, 10 (различия в пределах разброса); скорость растёт слабо (151 → 125 эпох). Переобучения нет: train MAE ≈ test MAE (0.0343 и 0.0341 при N = 10). Минимального N = 1 достаточно: в обученной сети

предактивации скрытых нейронов лежат в [−0.82; 0.67], где σ(s) практически линейна (корреляция s и σ(s) > 0.999), и один нейрон с линейным выходом воспроизводит рекуррентность ряда.

3. Почему на выходе используется линейная функция, а не сигмоида? Что было бы при сигмоиде и ненормализованных данных?

Прогнозирование – регрессия: выход должен принимать любые значения, а сигмоида ограничена интервалом (0; 1) и не может выдать значение вне него. При сигмоидном выходе и ненормализованных данных ($x \in [-2.09; 2.08]$) цели вне (0; 1) были бы недостижимы, а большие по модулю входы насыщали бы скрытые нейроны ($\sigma' \to 0$) и тормозили обучение.

4. Почему нормализация должна опираться только на обучающий участок? К чему приводит выход тестовых значений за [0; 1]?

Min/max по всему ряду передают в предобработку информацию о тесте (утечка данных) и завышают качество; в реальной задаче будущее неизвестно. Значения вне [0; 1] означают, что вход вышел за диапазон обучения, и прогноз становится ненадёжным. На наших данных тестовые значения лежат в [0.0056; 0.9846] (вне [0; 1] – 0 из 60): ряд периодичен, и обучающий участок покрывает всю амплитуду. Эффект проявился на окне с выбросом: вход 1.3689 дал ошибку 2.04.

5. Насколько устойчиво обучение к инициализации? Были ли запуски, не достигшие Ee, и от чего это зависит?

Все запуски с $p \geq 4$ достигли Ee (0/5 неудач для каждого $p \geq 4$ и каждого N); разброс малый (эпохи при p = 6: 129.8 ± 4.3, test MAE ± 0.001). Не достигли Ee только запуски с p = 2 (5 из 5) – причина в недостаточном окне, а не в инициализации: Es застревает на плато $\approx 3 \cdot 10^{-3}$. От η зависит лишь скорость (η = 0.01 – 1257 эпох, η = 0.3 – 50), N на сходимость почти не влияет.

6. Как растёт ошибка рекурсивного прогноза с увеличением горизонта? Сравните с одношаговым. До какого горизонта прогноз пригоден?

Предсказания подставляются в окно как входы, поэтому ошибки шагов накапливаются. На 30 шагах MAE = 0.0533 против 0.0213 у одношагового прогноза на тех же точках (в 2.5 раза выше), максимум 0.122 – порог 5 % размаха (0.209) не превышен; рост немонотонный. На 60 шагах средняя ошибка во второй половине вдвое выше (0.1095 против 0.0533), порог превышен на шаге 52. По критерию 5 % прогноз пригоден примерно до 50 шагов.

7. Как различие ошибок на обучении и тесте согласуется с обобщающей способностью? Какие меры улучшили бы обобщение? Что показал эксперимент с шумом?

Ошибки на обучающем и тестовом участках практически совпадают (MAE 0.0348 и 0.0339; sMAPE 9.99 % и 9.76 %) – сеть обобщает хорошо: ряд детерминирован, закон на тесте тот же. При переобучении train-ошибка была бы заметно ниже test-ошибки; улучшить обобщение могли бы ранняя остановка, уменьшение N, добавление шума, увеличение выборки. В эксперименте с шумом (σ = 2 % размаха) test MAE на чистом ряде 0.0779 (N = 5) и 0.0759 (N = 20) против 0.0050 и 0.0039 без шума: шум ухудшил точность, но подгонки шума нет (test − train = −0.0015), большее N обобщение не ухудшило.

Вопросы ЛР №1

1. Смог ли однослойный персептрон решить XOR? Объясните с точки зрения линейной разделимости.

Нет: за 10 000 эпох Es = 1.065, accuracy 0.50. Точки классов расположены по диагоналям квадрата, никакая прямая их не разделяет – задача линейно неразделима, а один нейрон строит только линейную границу.

2. Смог ли МСП 2-2-1 решить задачу? За сколько эпох?

Да: Es ≤ 0.01 достигнуто за 3566 эпох (Es = 0.009994), accuracy 1.00.

3. Как изменение диапазона выходных значений повлияло на обучение и выбор активации выходного слоя?

Сигмоида выдаёт значения только из (0; 1), поэтому c0 = 5 и c1 = −2 кодировались в 1 и 0, а выход декодировался обратно. После кодирования задача – обычная бинарная классификация, сигмоида на выходе подходит; для выдачи самих чисел нужен линейный выход.

4. Почему для XOR необходим хотя бы один скрытый слой?

Без него граница – прямая. Скрытый слой нелинейно преобразует пространство признаков, и классы становятся линейно разделимы для выходного нейрона.

5. Как результаты соотносятся с классическим XOR? Изменилась ли сложность?

Не изменилась: после кодирования c0/c1 → 1/0 задача тождественна классическому XOR; значения c0 и c1 влияют только на кодирование меток.

Вопросы ЛР №2

1. Какая функция потерь (MSE или BCE) сходится быстрее и устойчивее? Согласуется ли с теорией?

BCE быстрее: 858–1230 эпох (ср. 995) против 2761–3447 (ср. 2970) у MSE, примерно в 3 раза. По устойчивости BCE не лучше: сошлось 3 из 5 против 4 из 5. Скорость согласуется с теорией: у связки BCE + сигмоида градиент $\hat{y} - t$ не содержит σ', насыщение выхода не тормозит обучение.

2. Как нормализация целевых значений повлияла на обучение по сравнению с обучением на числах из [−10; 10]?

В нашей реализации нормализация c0/c1 → 0/1 использовалась в обеих конфигурациях и уже в ЛР №1, поэтому напрямую на числах сеть не обучалась. Без неё сигмоидный выход не может выдать 5 и −2; на скорость и устойчивость влияет только функция потерь.

3. Какова точность восстановления шкалы [c0; c1] у BCE и у MSE? Есть ли компромисс?

BCE точнее: MAE 0.0862 против 0.3471 у MSE (в 4 раза). Компромисса нет – BCE лучше по обоим показателям; результат зависит от порогов Ee (0.05 для BCE, 0.01 для MSE).

4. Насколько устойчивы конфигурации к инициализации? Были ли случаи недостижения критерия?

Устойчивость неполная: MSE сошёлся в 4 из 5 запусков, BCE – в 3 из 5. Остальные запуски застряли на плато (Es ≈ 0.52 у MSE, ≈ 1.53 у BCE; accuracy 0.75).

5. Подтвердилась ли линейная неразделимость XOR независимо от c0 и c1?

Да: после кодирования структура XOR не зависит от значений c0 и c1 – классы по диагоналям квадрата, линейно неразделимы.

6. Как выбор функции потерь соотносится с необходимостью многослойной архитектуры?

Не влияет: вывод определяется архитектурой. MSE и BCE на MLP 2-2-1 решают XOR, однослойный персептрон не решит ни с одной из них.

7. Что показывает разделяющая поверхность для обеих конфигураций?

Обе дают одну XOR-структуру – диагональную полосу высоких значений через (5, 5) и (−2, −2), образованную двумя полуплоскостями скрытого слоя. Различие лишь в резкости переходов: у BCE выходы ближе к крайним значениям шкалы.

Вопросы ЛР №3

1. Какая активация (сигмоида или ReLU) сходится быстрее? Более устойчива?

ReLU быстрее: 90 эпох (единственный сошедшийся запуск) против 858–1230 у сигмоиды. Устойчивее сигмоида: сошлось 3 из 5 против 1 из 5.

2. Как отличается форма разделяющей поверхности? Видны ли кусочно-линейные границы у ReLU?

Общая форма одинакова – диагональная XOR-структура. У ReLU переход между зонами уже и резче (кусочно-линейные границы), у сигмоиды плавнее; выходная сигмоида частично сглаживает кусочную линейность.

3. Почему ReLU предпочитают сигмоиде в глубоких сетях? Подтверждается ли на XOR?

Производная ReLU равна 1 при $x > 0$ и не затухает, а $\sigma' \leq 0.25$, и в глубоких сетях произведение таких множителей обнуляет градиент; ReLU к тому же дешевле. На XOR подтвердилось ускорение (90 против 858 эпох), но проявилась и неустойчивость из-за малого числа нейронов (всего 2).

4. Изменилась ли способность сети решать XOR при переходе на ReLU?

Нет, она определяется архитектурой. Оба варианта при благоприятной инициализации дают accuracy 1.00, но ReLU реализует её менее надёжно.

5. Есть ли «мёртвые» нейроны при ReLU? Как это повлияло на обучение?

Да: в 4 из 5 запусков один из двух скрытых нейронов «умер» (ReLU = 0 на всех примерах, градиент нулевой, веса не обновляются). Остался один рабочий нейрон, XOR не решается: Es застыла на ≈ 2.089, accuracy 0.75.

6. Как выбор активации соотносится с необходимостью многослойной архитектуры?

Активация не заменяет скрытый слой: нелинейность обязательна, а её вид влияет лишь на скорость и надёжность обучения.

7. Что показывает визуализация разделяющей поверхности? Совпадает ли граница с ожидаемой для XOR?

Обе конфигурации выучили одну и ту же XOR-границу (диагональ через (5, 5) и (−2, −2)), совпадающую с ожидаемой; у ReLU границы резче.

Вывод: на основе многослойного персептрона p–N–1 решила задачу прогнозирования временного ряда. Научилась формировать обучающую выборку методом скользящего окна, нормализовать данные, оценивать качество одношагового и многошагового прогноза и исследовать влияние размера окна p и числа нейронов скрытого слоя N на точность прогноза и скорость обучения.